Parsers and Grammars

Guillermo Polito

guillermo.polito@inria.fr
@guillep













Goals

- Understanding the Parsing Phase

- Grammars as language descriptions
- Grammar kinds

- Parser Implementation Flavours. E.g., Parser generators, combinators, handwritten

About Languages

- In these complex times, we communicate with machines using natural language (think e.g., ChatGPT). Let’s leave this out of scope :)

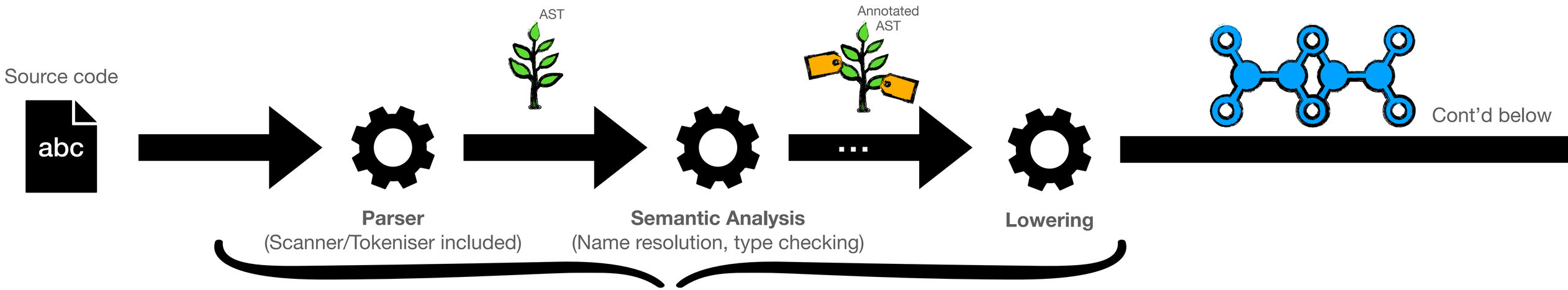
- However, programming still happens in *programming languages*
  - simpler rules than natural languages
  - stricter
  - ideally, non ambiguous

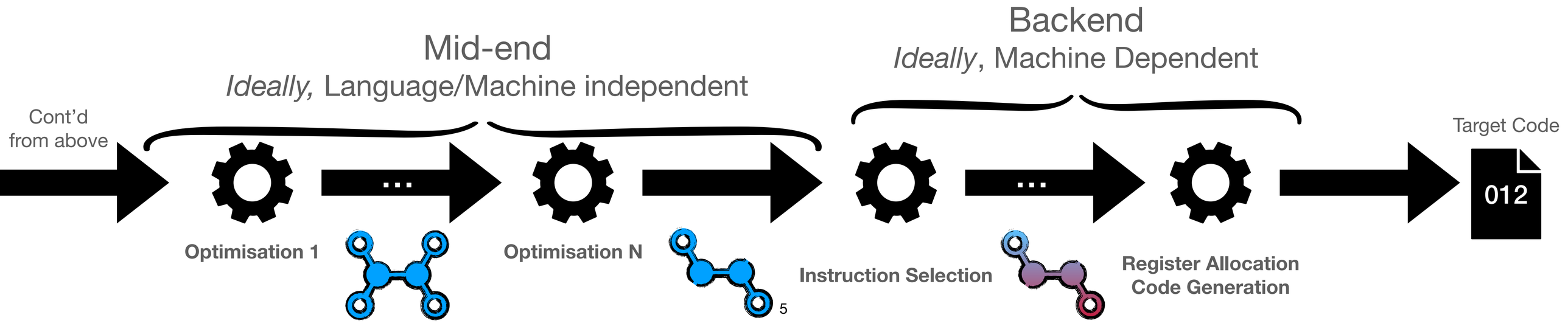
About Languages

- In these complex times, we communicate with machines using natural language (think e.g., ChatGPT). Let’s leave this out of scope :)

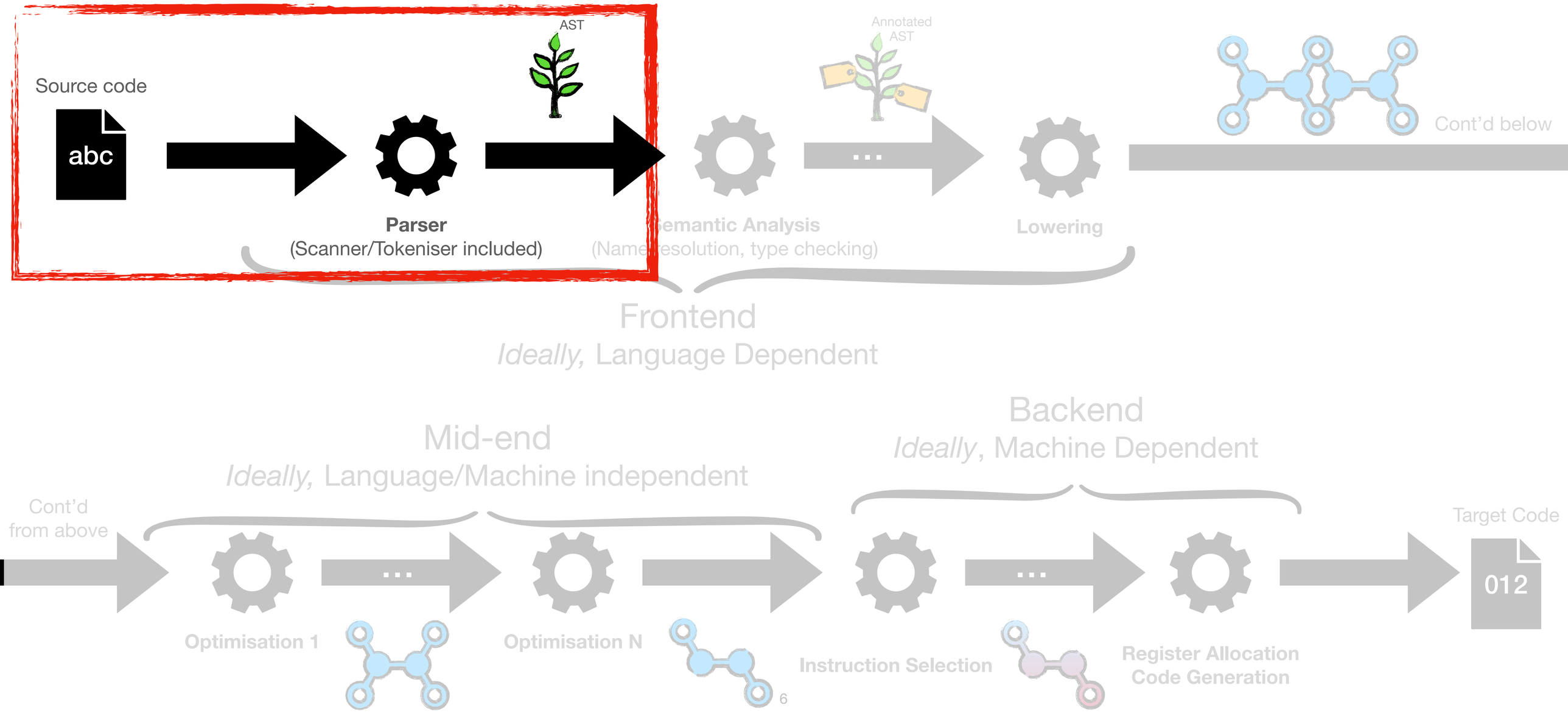
- However, programming still happens in *programming languages*
  - simpler rules than natural languages
  - stricter
  - ideally, non ambiguous
  - **And typically textual**

Modern Optimizing Compiler Phases

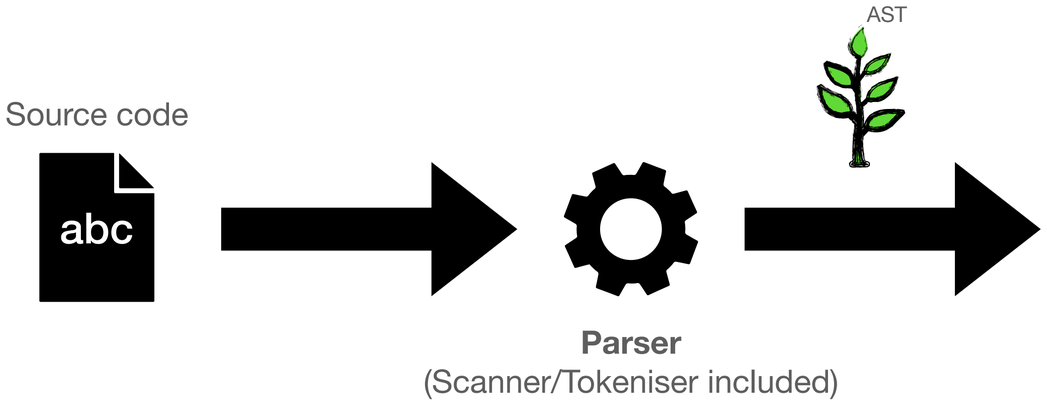




The Parsing Phase



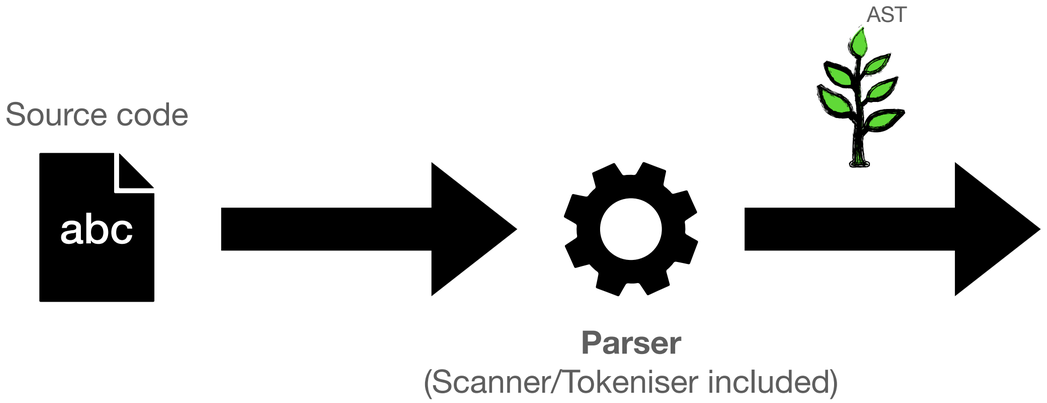
Why Parsing?



- Humans produce *text programs* (though there are other cooler ways to code)
- We need to make human text understandable to machines

- ***Parsers***: programs that interpret text of a programming language
- Two main responsibilities:
  - **Matching:** checks that the text follow the rules of the language
  - **Productions:** actions to execute when a match happens

Why Parsing?

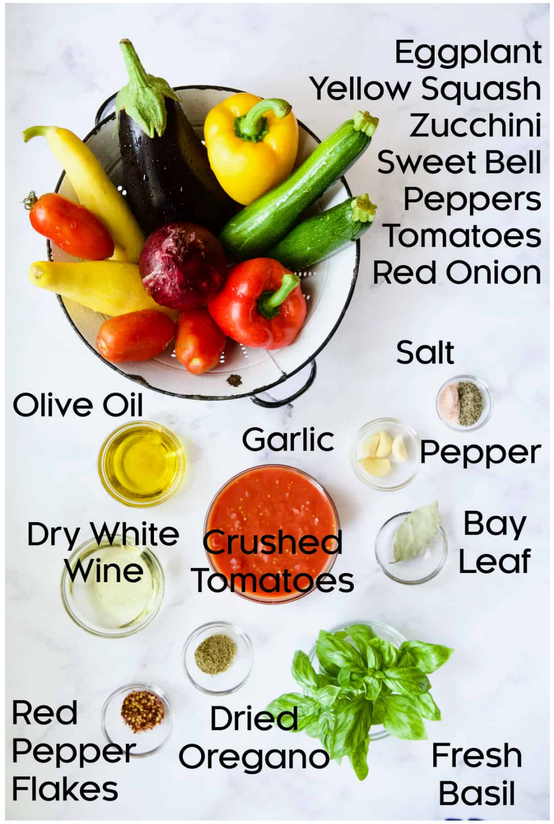


- Humans produce *text programs* (though there are other cooler ways to code)
- We need to make human text understandable to machines

- ***Parsers***: programs that interpret text of a programming language
- Two main responsibilities:
  - **Matching:** checks that the text follow the rules of the language
  - **Productions:** actions to execute when a match happens

Grammars

Parsers and Grammars



Your Program: the ingredients

Parser: the cook





Grammar: the recipe



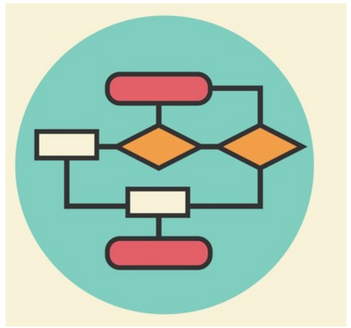
Something the machine can eat.

Parsers and Grammars

```
(3 / (a + b))
   . ceiling()
```

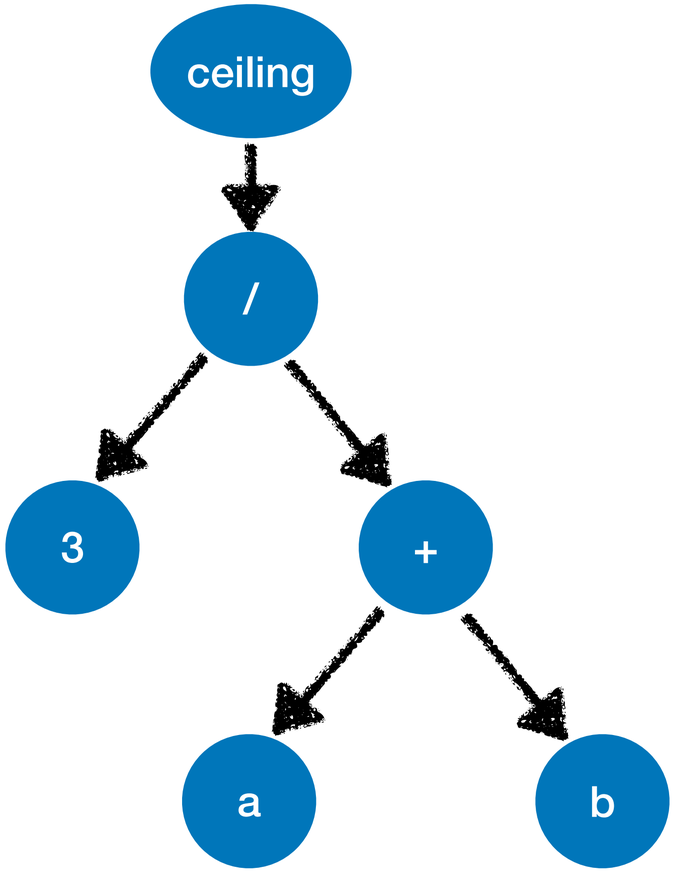
Your Program: the ingredients

Parser: the cook



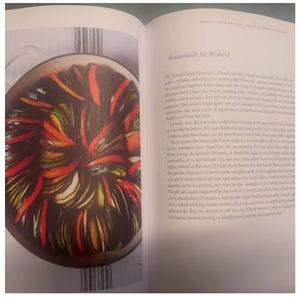
```
expression ::=
   expression binop expression
expression ::=
   expression.message(args)
…
```

Grammar: the recipe



Something the machine can eat.
A data structure

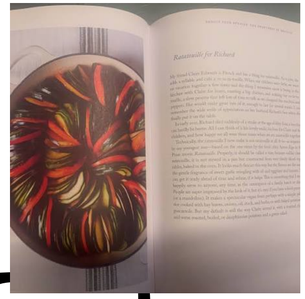
Grammars



- *Formal* descriptions of a language
- Used in formal language theory. Take a look at Chomsky!

- Made of
  - a set of **terminal values**, the *atomic elements* of our language
  - a set of (derivation/production) **rules** on how they *combine*

An Integer Number Grammar



- Example, a number grammar

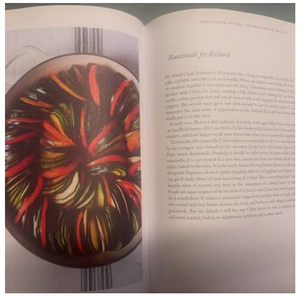
```
ntNumber --> ntDigit, ntNumber | ntDigit.
ntDigit --> ($0 - $9).
```

a number is either:
- a digit
- or a digit followed by a number

a digit is a character ranging between 0 and 9

- **Terminals:** the values between 0 and 9
- **The rules:** what is a number, what is a digit

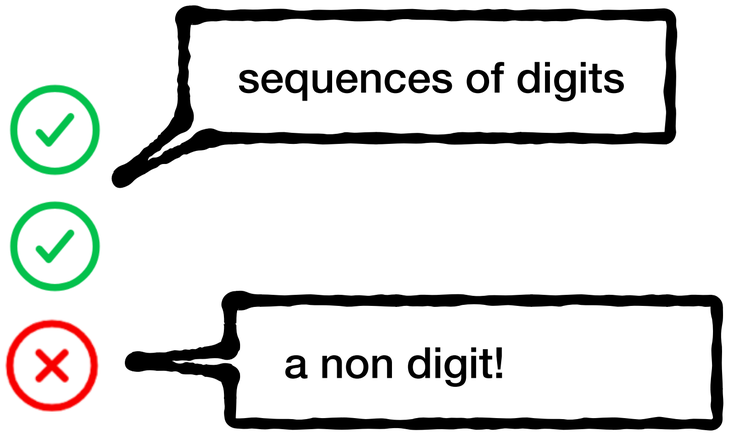
Matching Is About Language Rules



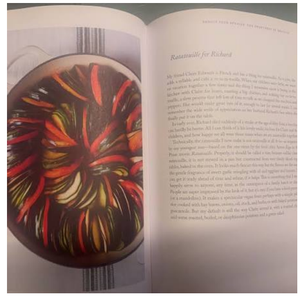
- Example, a number grammar

```
ntNumber --> ntDigit, ntNumber | ntDigit.
ntDigit --> ($0 - $9).
```

```
MyParser matches: '0'.
MyParser matches: '99'.
MyParser matches: '+'.
```



Matching Is About Language Rules



- Example, a number grammar

```
ntNumber --> ntDigit, ntNumber | ntDigit.
ntDigit --> ($0 - $9).
```

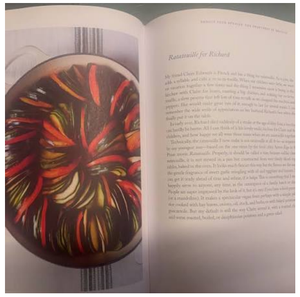
```
MyParser matches: '0'.
MyParser matches: '99'.
MyParser matches: '+'.
```

```
MyParser matches: ''.
MyParser matches: '2000000000000000000'.
```

the empty text

a 65-bit number matches, can we put it in memory?

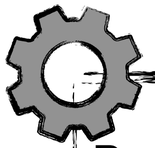
Production Rules



- Example, a number grammar

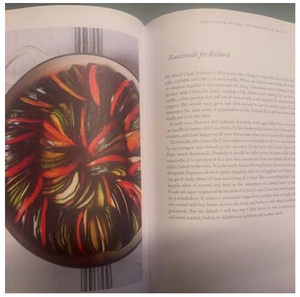
```
ntNumber --> ntDigit, ntNumber | ntDigit.
ntDigit --> ($0 - $9).
```

+



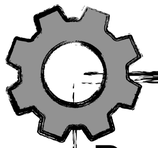
Production Rule:
for each match
-> generate a parse tree node

Production Rules



- Example, a number grammar

```
ntNumber --> ntDigit, ntNumber | ntDigit.
ntDigit --> ($0 - $9).
```

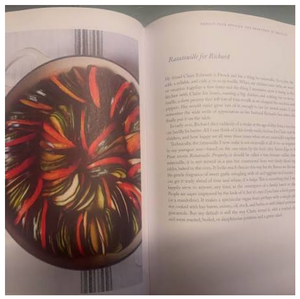


Production Rule:
for each match
-> generate a parse tree node

```
MyParser parse: '0'.
```

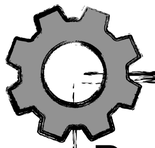


Production Rules



- Example, a number grammar

```
ntNumber --> ntDigit, ntNumber | ntDigit.
ntDigit --> ($0 - $9).
```

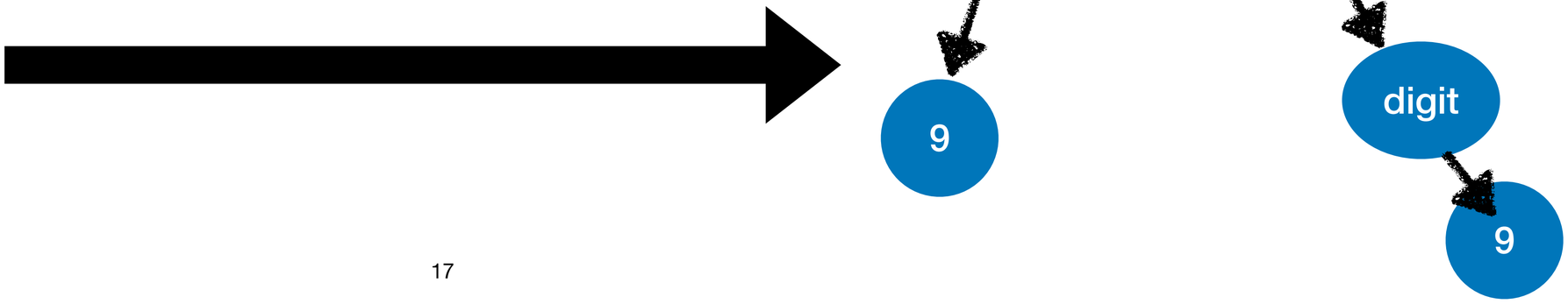


Production Rule:
for each match
-> generate a parse tree node

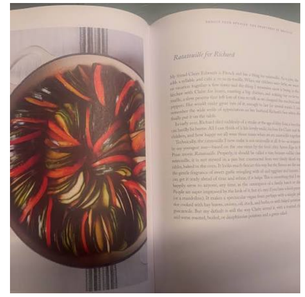
```
MyParser parse: '0'.
```




```
MyParser parse: '99'.
```

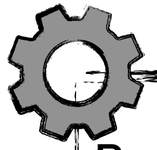


Production Rules



- Example, a number grammar

```
ntNumber --> ntDigit, ntNumber | ntDigit.
ntDigit --> ($0 - $9).
```



Production Rule:
for each match
-> generate a parse tree node

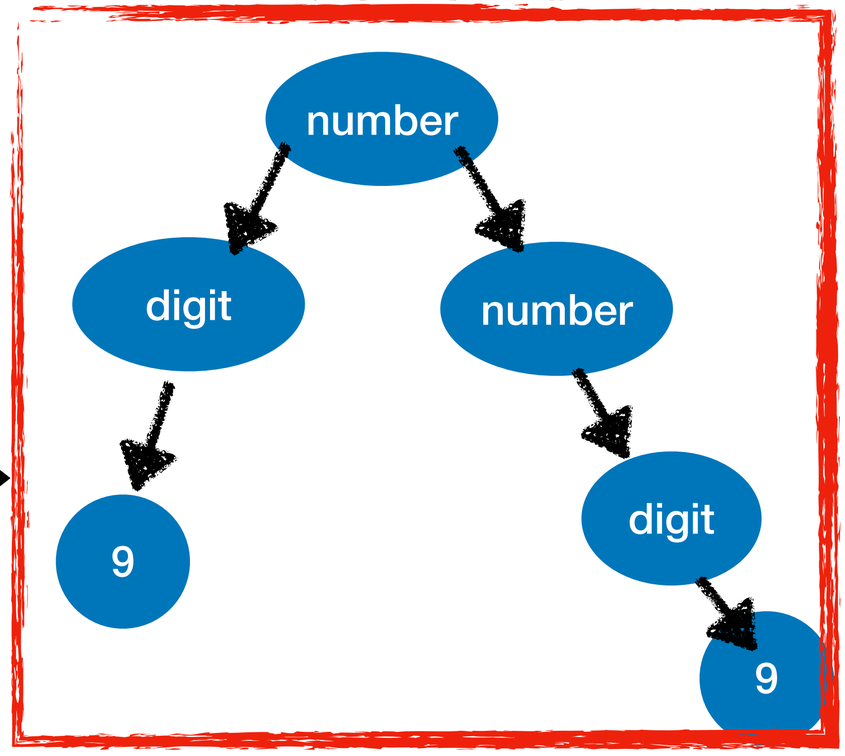
```
MyParser parse: '0'.
```



Complex Parse Trees…

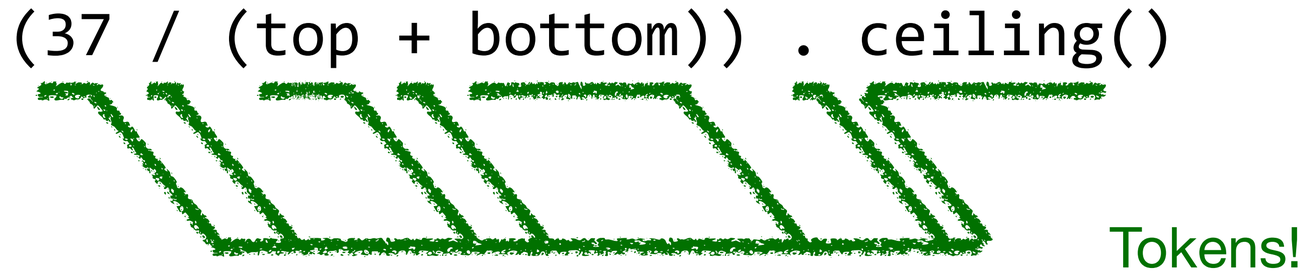
```
MyParser parse: '99'.
```





Tokenization and Parsing

- Parsing is typically separated into two steps
  - **Lexing/Scanning/Tokenization** breaks into small meaningful chunks
  - *Proper* **Parsing** makes sense of those small chunks



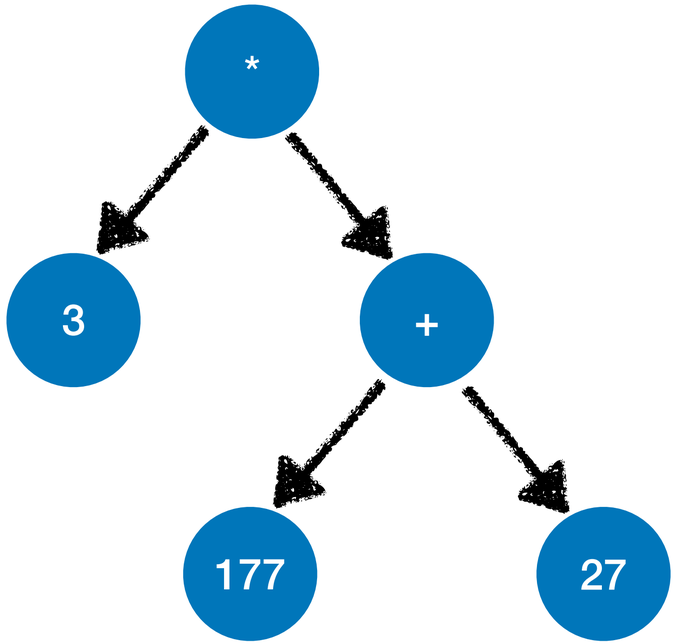
Grammars Distinguish Tokens

```
Complex Token Rules:
  ntNumber --> ntDigit, ntNumber | ntDigit.
  ntDigit --> ($0 - $9).

Grammar Main Rules:
  ntExpression --> ntNumber ntBinOp ntNumber
  ntBinOp --> '+' | '*'
```

```
MyParser parse: '3 * 177 + 27'.
```





A cleaner parse tree

Common Parsing Strategies

- **Top-down** parsing *e.g., LL parsers, recursive descent*
  - start working from the *top of the tree*
  - apply derivation rules *left-to-right*
- **Bottom-up** parsing *e.g., LR parsers*
  - start detecting the leaves of the tree
  - apply derivations up, matching the terminals



Top-down vs Bottom-up Parsing

- Top-down parsing
  - Conceptually **simple**, and simple to implement!
  - **Less efficient** if the grammar is ambiguous: *lookahead or backtracking*
  - Suffer from **ambiguities**: grammars need to be adapted
- Bottom-up parsing
  - **Complex** implementations
  - Handle **larger set of languages**
  - **Resolves ambiguities better**

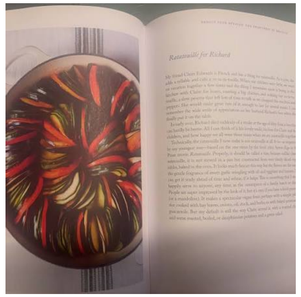
Popular Parser Implementation Strategies

- **Parser Generators:** generate standalone a parser from a grammar
- **Parser Combinators:** *functional-style*, parsers are composed functions
- **Handwritten** recursive descent parsers: in 2023, 7 of the top 10 languages!!

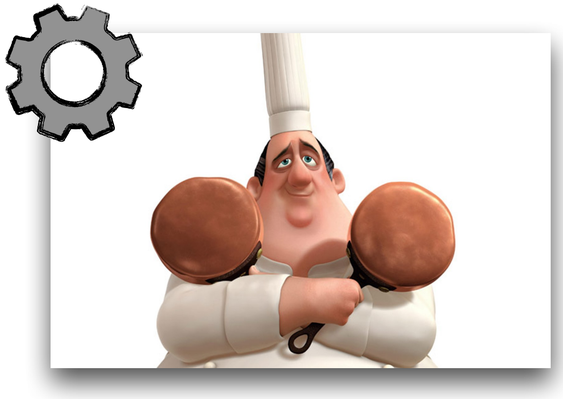
https://kddnewton.com/2023/06/12/rewriting-the-ruby-parser.html

Parser Generators

- e.g., *yacc*



A grammar



A parser generator



A parser specialized for our grammar

Parser Combinators

- *e.g., Haskell parser combinators, PetitParser*



Small parsers, each *encodes* a part of the grammar

combined



Combined form a full parser

Handwritten Parsers



I (human) know the grammar, I (human) code the parser



A parser specialized for our grammar

https://kddnewton.com/2023/06/12/rewriting-the-ruby-parser.html

Glimpse at Parser Implementation Strategies

- **Parser Generators:**
  - Typically bottom-up
  - We just need a grammar!
  - Difficult to *tweak/adapt*
- **Parser Combinators and Handwritten**
  - Typically top-down
  - The grammar is embedded, not explicit
  - Easier to tweak and adapt

Takeaways

- Grammars and parsers form a large and well studied topic
- Used to recognize languages, and more often
  - to produce code of data structures representing the code (e.g., ASTs)

- Practical solutions: combinators, generators, handwritten

Material

- Most compiler books
- Laurence Tratt discussion on mainstream language parsers
  - https://tratt.net/laurie/blog/2020/which_parsing_approach.html
  - https://tratt.net/laurie/blog/2023/why_we_need_to_know_lr_and_recursive_descent_parsing_techniques.html
- The Fuzzing Book. Grammars Chapter. A. Zeller et al https://www.fuzzingbook.org/html/Grammars.html